



Faculdades de Campinas

ENGENHARIAS DA COMPUTAÇÃO

Tratamento de dados e armazenamento de informações - Aula 2

Henrique Palorca

Felipe Furlan

Henrique Biruel

Campinas

Agosto de 2026

PROMPT UTILIZADO:

Contexto

Sou aluno de Engenharia de Computação da FACAMP, na disciplina Tratamento e Armazenamento da Informação. Estou anexando o material da aula temática "SQL e NoSQL" (animação interativa com 56 slides, do professor da disciplina) e preciso responder as 10 questões dissertativas que estão no slide 54.

Tarefa

Responda as 10 questões dissertativas, uma a uma, em texto corrido de nível universitário.

Regras de conteúdo

1. Use o material anexado como referência primária. Vocabulário, exemplos e critérios de decisão devem ser coerentes com o que os slides apresentam.
2. Quando um termo tiver definição fechada no material, use essa definição e não a genérica. Exemplo: "contexto de utilização" é definido nos slides 46, 47 e 49 como um dos cinco contextos do simulador (OLTP; analytics e BI; catálogo e documentos; tempo real e telemetria; IA e RAG).
3. Antes de responder, aponte qualquer ambiguidade que você encontrar nos enunciados e diga qual interpretação vai adotar e por quê.
4. Verifique na web tudo que envelhece: versões, licenças de software, certificações e serviços de nuvem. Se o material estiver desatualizado em algum ponto, sinalize isso separadamente, com a fonte, em vez de repetir a informação antiga na resposta.
5. Todo exemplo de código (SQL, CQL, Cypher, MongoDB) deve ser sintaticamente válido e executável. Indique o dialeto ou banco em que ele roda.
6. Não invente referências bibliográficas, números, benchmarks ou nomes de produto. Se não tiver certeza de um dado, diga que não tem.
7. Nas questões de comparação (1, 2, 6, 9), apresente também as limitações e o custo da abordagem que parecer mais favorável. Não escreva propaganda de tecnologia.
8. Na questão 4, use um cenário de negócio realista e mostre as tabelas envolvidas antes das consultas.
9. Na questão 10, deixe explícito qual segmento e quais contextos foram escolhidos, e feche com uma comparação entre as duas arquiteturas.

Formato

- Português do Brasil, registro acadêmico, primeira pessoa evitada.
- Numeradas de 1 a 10, com um título curto em cada.
- Entre 200 e 350 palavras por questão, texto dissertativo corrido. Use lista apenas quando o conteúdo for realmente enumerável.
- Sem formatação pesada de markdown, porque o texto será colado em um documento.

Ao final

Liste em uma seção separada: (a) as inconsistências, imprecisões ou informações desatualizadas que você encontrou no material anexado, com o número do slide; (b) os pontos das suas respostas em que você teve menos confiança e que eu deveria conferir antes de entregar.

QUESTÕES DISSERTATIVAS-BANCOS DE DADOS

1) Diferenças entre ACID e BASE

ACID (Atomicidade, Consistência, Isolamento, Durabilidade) é o conjunto de garantias das transações em bancos relacionais: ou a transação inteira acontece ou nada acontece; o banco sai de um estado válido para outro estado válido; transações concorrentes não enxergam estados intermediários umas das outras; e o que foi confirmado sobrevive a queda de energia ou falha do servidor. O preço disso é coordenação: locks, controle de concorrência e, em ambiente distribuído, protocolos como o commit em duas fases, que aumentam latência e dificultam a escala horizontal.

BASE (Basically Available, Soft state, Eventually consistent) é a abordagem oposta, típica de sistemas distribuídos NoSQL. Pelo teorema CAP, quando há partição de rede é preciso escolher entre consistência e disponibilidade; o BASE escolhe disponibilidade. O sistema continua respondendo mesmo com nós fora do ar, aceita que réplicas fiquem temporariamente divergentes e converge para o mesmo valor depois de um tempo.

Quando usar cada um: ACID é adequado sempre que existe invariante forte e o erro custa dinheiro ou responsabilidade legal-movimentação financeira, baixa de estoque, emissão fiscal, folha de pagamento, reserva de assento, ERP em geral. BASE é adequado quando a disponibilidade e o volume valem mais do que a exatidão instantânea-feed de rede social, contador de visualizações, catálogo de produtos, telemetria e IoT, logs, cache, histórico de navegação. Vale registrar que hoje a divisão é menos binária: o MongoDB tem transações ACID multi-documento e o Cassandra tem consistência ajustável por consulta e transações leves.

2) Modelo relacional x modelo documento

O modelo relacional organiza os dados em tabelas com esquema fixo, chaves primárias e estrangeiras, normalização para eliminar redundância e SQL como linguagem de consulta. Vantagens: integridade referencial garantida pelo próprio banco, ausência de duplicação, consultas ad-hoc muito flexíveis (é possível cruzar tabelas que nunca se pensou em cruzar), transações ACID e ecossistema maduro de ferramentas. Limitações: o esquema rígido torna mudanças custosas em bases grandes, existe o "impedance mismatch" entre tabelas e os objetos da aplicação, JOINS pesados degradam em volumes muito altos e a escala horizontal é trabalhosa.

O modelo documento (MongoDB, Couchbase) guarda registros auto-contidos em JSON/BSON, com esquema flexível e dados aninhados. Vantagens: o documento se parece com o objeto que a aplicação já manipula, um agregado inteiro é lido em uma única operação sem JOIN, campos novos podem ser adicionados sem migração e o sharding é nativo. Limitações: a desnormalização gera duplicação, e

manter a cópia coerente vira responsabilidade da aplicação; consultas que cruzam coleções são limitadas; documentos podem crescer descontroladamente; e a ausência de esquema imposto pelo banco muitas vezes vira dado sujo.

Exemplos de aplicação: relacional para ERP, financeiro, faturamento, controle de estoque e qualquer domínio com muitos relacionamentos e relatórios. Documento para catálogo de produtos com atributos que variam por categoria, CMS e conteúdo editorial, perfil de usuário, registro de eventos e protótipos em que o modelo ainda está mudando toda semana.

3) Papel dos índices em SQL e NoSQL

Um índice é uma estrutura auxiliar que mapeia valores de uma ou mais colunas para a localização física dos registros, permitindo que o banco encontre as linhas desejadas sem varrer a tabela inteira. Em bancos SQL o padrão é a árvore B (B-tree), boa para igualdade, faixa e ordenação; existem ainda hash, GIN e GiST para texto e JSON, BRIN para tabelas enormes ordenadas por data, e índices únicos que implementam restrições de integridade. Índices aceleram WHERE, JOIN, ORDER BY, DISTINCT e agregações.

Em NoSQL o conceito aparece de formas diferentes. No MongoDB há índices simples, compostos, parciais, TTL, de texto e geoespaciais, além do índice implícito da chave primária. No Cassandra a modelagem é dirigida pela consulta: a chave de partição e as chaves de clustering são o índice principal, e índices secundários são desaconselhados em colunas de alta cardinalidade porque forçam consulta em todos os nós. No Redis não há índice automático-o desenvolvedor constrói o "índice" à mão com estruturas como sorted sets e conjuntos.

O excesso de índices se torna problema porque cada INSERT, UPDATE e DELETE precisa atualizar todos os índices afetados, tornando a escrita mais lenta; porque cada índice ocupa disco e disputa memória de cache com os dados realmente quentes; porque um otimizador com opções demais pode escolher um plano ruim; e porque índices redundantes ou nunca usados ainda assim consomem manutenção, backup e tempo de reconstrução. A prática correta é indexar a partir das consultas reais, monitorar o uso (por exemplo, pg_stat_user_indexes no PostgreSQL), remover os ociosos e preferir poucos índices compostos bem ordenados a muitos índices de coluna única.

4) Exemplo de JOIN e de agregação com GROUP BY

Cenário: sistema de vendas de uma rede varejista, com as tabelas pedido, item_pedido, cliente, vendedor, produto e categoria.

JOIN-listar os pedidos do ano com o nome do cliente e do vendedor responsável:

```
SELECT p.id,  
       c.nome AS cliente,  
       v.nome AS vendedor,  
       p.data_emissao,  
       p.valor_total
```

```

FROM pedido p
JOIN cliente c ON c.id = p.cliente_id
LEFT JOIN vendedor v ON v.id = p.vendedor_id
WHERE p.data_emissao >= '2026-01-01'
ORDER BY p.data_emissao DESC;
```

```

O JOIN com cliente é interno porque todo pedido obrigatoriamente tem cliente; o LEFT JOIN com vendedor preserva os pedidos feitos pelo site, que não têm vendedor associado-trocar por INNER JOIN aqui esconderia silenciosamente parte do faturamento.

GROUP BY-faturamento mensal por categoria de produto:

```

```
SELECT cat.nome AS categoria,
       DATE_TRUNC('month', p.data_emissao) AS mes,
       COUNT(DISTINCT p.id) AS qtd_pedidos,
       SUM(ip.quantidade * ip.preco_unitario) AS faturamento,
       AVG(ip.quantidade * ip.preco_unitario) AS ticket_medio_item
FROM pedido p
JOIN item_pedido ip ON ip.pedido_id = p.id
JOIN produto pr ON pr.id = ip.produto_id
JOIN categoria cat ON cat.id = pr.categoria_id
WHERE p.status = 'FATURADO'
GROUP BY cat.nome, DATE_TRUNC('month', p.data_emissao)
HAVING SUM(ip.quantidade * ip.preco_unitario) > 10000
ORDER BY mes, faturamento DESC;
```

```

A consulta transforma milhões de itens em uma tabela de decisão para o gestor de compras. Vale destacar a diferença entre WHERE e HAVING: o WHERE filtra as linhas antes do agrupamento (só pedidos faturados entram na conta) e o HAVING filtra depois, sobre o resultado já agregado (só categorias que passaram de dez mil reais no mês).

## 5) Sharding e escalabilidade horizontal

Sharding é o particionamento horizontal dos dados entre vários nós: cada shard guarda um subconjunto das linhas ou documentos, e não uma cópia do todo. A distribuição é definida pela shard key, que pode ser aplicada por hash (distribuição uniforme, mas consultas por faixa precisam perguntar a todos os nós), por faixa de valores (ótimo para intervalos, com risco de hotspot quando os dados novos caem sempre na mesma partição) ou por lista, região geográfica ou cliente/tenant.

A relação com a escalabilidade horizontal é direta. A escala vertical-colocar mais CPU, memória e disco na mesma máquina-tem teto físico e custo que cresce de forma não linear. O sharding permite crescer somando máquinas comuns, distribuindo não só o volume de dados, mas também IOPS, memória e throughput de escrita. É importante distinguir de replicação: réplicas escalam leitura e dão

alta disponibilidade, mas todas guardam a base inteira e não resolvem o gargalo de escrita nem o tamanho do dado.

O custo do sharding é operacional e de projeto: a escolha da shard key é praticamente irreversível e uma escolha ruim gera nós sobrecarregados; consultas e JOINS que cruzam shards viram scatter-gather e ficam caros; transações distribuídas exigem coordenação; e rebalancear ou adicionar nós é uma operação delicada. MongoDB, Cassandra e Elasticsearch fazem sharding nativamente; no PostgreSQL costuma-se usar particionamento declarativo, extensões como o Citus ou lógica na aplicação.

## **6) Critérios para escolher entre PostgreSQL, MongoDB, Redis e Cassandra**

Os critérios que devem orientar a decisão são: a natureza do dado e a densidade de relacionamentos; o padrão de acesso (consultas conhecidas de antemão ou exploração ad-hoc); o requisito de consistência e de transação; volume atual e curva de crescimento; latência aceitável; distribuição geográfica; necessidade de durabilidade; maturidade e tamanho do time; e custo total de operação. Vale lembrar que a escolha não precisa ser única-persistência poliglota, com cada banco cuidando do que faz melhor, costuma ser a resposta real.

PostgreSQL: dados com muitos relacionamentos, invariantes fortes, transações, integridade fiscal e financeira, relatórios ad-hoc. Cobre também JSONB para trechos semiestruturados e pgvector para busca semântica, o que faz dele o default sensato para a maioria dos projetos novos.

MongoDB: quando o dado natural é um agregado auto-contido, o esquema varia entre registros ou muda com frequência, e o time quer velocidade de evolução-catálogos, conteúdo, perfis, documentos de negócio.

Redis: quando o requisito é latência de submilissegundo e o dado é volátil ou reconstruível-cache, sessão, carrinho, rate limiting, ranking, lock distribuído, fila e pub/sub. Por ser em memória, é complemento e raramente fonte da verdade.

Cassandra: escrita massiva e contínua, séries temporais, eventos, logs e IoT, com necessidade de operar em múltiplos datacenters e nunca ficar indisponível. Exige que as consultas sejam conhecidas antes da modelagem, não faz JOIN e trabalha com consistência ajustável.

Regra prática: começar pelo relacional e introduzir os demais quando o gargalo estiver medido, não presumido.

## **7) Arquitetura mínima de dados para uma aplicação de RAG**

Uma aplicação de RAG (Retrieval-Augmented Generation) precisa de três camadas de dados com papéis distintos.

Banco relacional-é a fonte da verdade operacional e de governança. Guarda usuários, organizações e permissões; o catálogo de documentos (identificador, origem, versão, hash, data de ingestão, dono, regras de acesso); o registro dos chunks e sua relação com o documento pai; o controle dos jobs de

ingestão; o histórico de conversas, o feedback do usuário e os logs de auditoria e custo. É ele que permite responder "quem pode ver o quê" e reprocessar o que falhou.

Índice vetorial-armazena os embeddings dos chunks junto com metadados filtráveis (documento, organização, data, tags) e executa busca aproximada por similaridade (ANN, com HNSW ou IVF) para trazer os k trechos mais próximos da pergunta. Pode ser o pgvector dentro do próprio PostgreSQL em escalas menores, ou um serviço dedicado como Qdrant, Weaviate ou Pinecone. Na prática, combinar a busca vetorial com busca léxica (BM25) e um reranker melhora bastante a precisão.

Armazenamento de documentos-guarda o arquivo original (PDF, DOCX, HTML) e o texto extraído e fatiado. Costuma ser object storage (S3, GCS) ou um banco de documentos. O índice vetorial guarda apenas o vetor e o ponteiro; o texto que vai efetivamente para o prompt é recuperado daqui, o que evita inchar o índice e permite reprocessar sem perder o original.

O fluxo é: ingestão → extração e limpeza → chunking → geração de embeddings → gravação no índice vetorial com metadados. Na consulta: embedding da pergunta → filtro por permissão e metadados → busca ANN → recuperação dos textos → reranking → montagem do prompt → resposta com citação da fonte. Pontos de atenção: versionar o modelo de embedding (trocar-lo obriga a reindexar tudo), garantir isolamento entre clientes no filtro, e usar um cache (Redis) para embeddings e respostas repetidas.

## **8) Riscos de segurança e LGPD no armazenamento de dados pessoais**

Os riscos técnicos mais comuns são acesso indevido por credenciais compartilhadas ou usuários com privilégio excessivo; injeção de SQL em aplicações mal construídas; backups e réplicas sem criptografia; cópia de base de produção para ambientes de desenvolvimento e teste sem mascaramento; dados pessoais gravados em logs e mensagens de erro; exportações em planilha que saem do perímetro controlado; instâncias expostas na internet com configuração padrão; ausência de trilha de auditoria, que impede saber quem acessou o quê; e retenção indefinida de dados que já cumpriram sua finalidade.

Do ponto de vista da LGPD, todo tratamento precisa de base legal e de finalidade específica, com minimização-não guardar o que não é necessário. Dados sensíveis (saúde, biometria, origem racial, convicção religiosa, opinião política) exigem cuidado redobrado. Os direitos do titular impõem exigências diretas ao modelo de dados: para atender pedidos de acesso, correção, portabilidade e principalmente eliminação, é preciso ser capaz de localizar todos os registros de uma pessoa-o que fica difícil quando o dado foi duplicado em bases NoSQL desnormalizadas, data lake, índices de busca, backups e logs. Somam-se as obrigações de controlar o compartilhamento com operadores e terceiros, notificar incidentes à ANPD e ao titular, e o risco de sanções que vão de advertência a multa e bloqueio dos dados.

Os controles esperados são criptografia em repouso e em trânsito com gestão adequada de chaves; controle de acesso baseado em papéis com privilégio mínimo; anonimização, pseudonimização ou mascaramento em ambientes que não são de produção; tokenização de identificadores como CPF e cartão; segurança em nível de coluna e uso de views para limitar exposição; auditoria de acessos;

política de retenção com expurgo automático; catálogo de dados com classificação de sensibilidade; e privacy by design considerado já na modelagem, e não como remendo posterior.

## **9) Ambiente analítico x ambiente transacional**

Modelagem: o ambiente transacional (OLTP) é normalizado, geralmente até a terceira forma normal, com entidades granulares e chaves estrangeiras, otimizado para escrever pouco dado por vez sem redundância e sem violar integridade. O ambiente analítico (OLAP) é deliberadamente desnormalizado, usando modelagem dimensional em estrela ou floco de neve: uma tabela fato com as métricas em um grão bem definido (por exemplo, item vendido) cercada por dimensões (cliente, produto, tempo, loja, canal), com dimensões de mudança lenta para preservar o histórico-saber qual era a categoria do produto na data da venda, e não a de hoje.

Armazenamento: o OLTP é orientado a linha, porque quase sempre lê ou grava o registro inteiro, e usa índices B-tree sobre dados recentes e quentes. O OLAP é colunar (Parquet, BigQuery, Redshift, ClickHouse), o que permite ler apenas as colunas usadas na consulta, comprimir muito melhor os dados repetidos e varrer volumes enormes; o particionamento por data e o clustering substituem boa parte do papel dos índices. A carga é feita em lote ou por streaming, através de processos de ETL/ELT, e o histórico é retido por anos.

Consultas: no OLTP são curtas, altamente seletivas, atingem poucas linhas por chave, rodam em milissegundos e com concorrência alta de muitos usuários simultâneos, com mistura de leitura e escrita. No OLAP são longas, varrem milhões de linhas, concentram-se em agregações, funções de janela e GROUP BY, atendem poucos usuários simultâneos e são quase totalmente de leitura, aceitando segundos ou minutos de resposta.

A separação também tem motivo operacional: rodar relatórios pesados no banco transacional compete por recursos e degrada a operação. Em troca, aceita-se uma latência do dado analítico, que pode ser de minutos a um dia.

## **10) Segmento escolhido: Varejo-arquitetura de banco de dados para dois contextos de utilização**

Observação de leitura da questão: no simulador da aula, "contexto" tem significado definido-são os cinco contextos de utilização (OLTP; analytics e BI; catálogo, conteúdo e documentos; tempo real, cache, eventos e telemetria; IA, RAG e busca semântica). A resposta abaixo escolhe o segmento Varejo e desenvolve dois desses contextos.

Caracterização do segmento (Varejo): dores de omnichannel, controle de estoque, promoções e forte sazonalidade; dados de clientes, produtos, preços, pedidos, carrinhos e cliques. É um segmento em que convivem, no mesmo negócio, um núcleo transacional pequeno e crítico e uma borda de altíssimo volume e baixo valor unitário.

### **CONTEXTO 1-Operação transacional (OLTP)**

Objetivo: registrar vendas, estoque, preços e pedidos com integridade forte e resposta em milissegundos, inclusive no PDV da loja.



Stack: PostgreSQL como banco principal (alternativa corporativa: SQL Server), Redis para cache de tabela de preços e sessão do PDV, réplica de leitura para relatórios operacionais.

Modelagem: entidades e processos críticos mapeados; chaves primárias e estrangeiras explícitas; normalização até a 3FN; índices seletivos criados a partir das consultas reais (busca de produto por código de barras, pedidos por cliente, movimentação por data).

Criação: criação do schema, tabelas, constraints e índices; carga de dados mestres (produtos, preços, lojas, fiscal); definição de roles, política de backup com recuperação a ponto no tempo e trilha de auditoria.

Armazenamento: row store com log transacional; replicação para alta disponibilidade; particionamento das tabelas de movimento por data quando o volume justificar; retenção legal dos documentos fiscais.

Justificativa: o domínio é intensamente relacional e cheio de invariantes que não podem quebrar—não se vende item sem baixar estoque, não se emite nota com valor divergente. Isso exige ACID, e não consistência eventual. O volume é moderado e cabe em uma instância bem dimensionada, então sharding aqui seria complexidade sem retorno. As consultas de gestão são ad-hoc e mudam com frequência, o que favorece SQL sobre modelos dirigidos pela consulta. O Redis entra apenas onde a latência importa e o dado é reconstruível, e a réplica evita que o relatório do gestor concorra com o caixa da loja.

## CONTEXTO 2-IA, RAG e busca semântica

Objetivo: permitir que o cliente busque por intenção ("tênis para corrida na chuva") e que o atendimento e o comprador da rede consultem em linguagem natural políticas, fichas técnicas e histórico, com respostas rastreáveis até a fonte.

Stack: PostgreSQL com pgvector para metadados e embeddings (alternativa em escala: Qdrant ou Milvus como índice dedicado), object storage para os documentos originais, OpenSearch/Elasticsearch para a busca léxica que compõe a consulta híbrida.

Modelagem: separação clara entre documento, chunk, embedding e metadados; política definida de atualização dos embeddings quando o catálogo muda; registro de permissões e classificação do conteúdo; plano de avaliação de relevância com feedback.

Criação: bucket de objetos e tabela de metadados; geração de chunks e embeddings; criação do índice vetorial e do índice lexical; publicação do pipeline de consulta híbrida com logging de prompts e respostas.

Armazenamento: objetos e texto extraído no object storage; vetores no índice; metadados e permissões no relacional; logs de inferência com retenção definida. Versionamento dos embeddings, porque trocar o modelo obriga a reindexar toda a base.

Justificativa: aqui o requisito não é integridade transacional, e sim precisão de recuperação e controle sobre o que a IA pode ver. O relacional continua presente, mas com outro papel-deixa de ser o executor da venda e passa a ser o guardião de metadados, permissões e rastreabilidade. Manter o pgvector dentro do próprio PostgreSQL reduz o número de tecnologias a operar enquanto o volume de documentos for moderado; a migração para um índice vetorial dedicado só se justifica quando a latência da busca aproximada virar gargalo medido. A busca híbrida existe porque, no varejo, parte das consultas é semântica e parte é literal (código do produto, SKU, marca), e só o vetor erraria as literais.

Comparação e conclusão: o mesmo segmento exige decisões opostas conforme o contexto. No OLTP, o critério dominante é consistência e o desenho é normalizado, row store, com uma tecnologia só. No contexto de IA, o critério dominante é recuperação relevante e governança, e o desenho é poliglota, com o dado desnormalizado em chunks e o relacional relegado ao papel de catálogo e controle de acesso. É exatamente o que a aula defende como persistência poliglota: não existe banco melhor em abstrato, existe aderência da tecnologia ao problema.